Software Requirements Specification

for

High-Concurrency Ledger Service

Version 2.0

Prepared by Group 4

<u>Date of creation:</u> 03/02/2026

Table of Contents

Table of Contents 2
Revision History 3
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 System Components 1
2. Overall Description 3
2.1 Transaction Processing 3
2.2 Data Management 3
2.3 Replication and Fault Tolerance 4
2.4 Distributed Coordination 5
3. Non-Functional Requirements 6
3.1 Performance 6
3.2 Reliability and Safety 6
3.3 Quality Attributes 6
4. System Architecture 7
4.1 Component Interactions 7
4.2 Data Flow 7
4.3 Component Responsibilities 8
4.4 UML Diagrams 9
5. Constraints and Assumptions 16
5.1 Performance Requirements 16
5.2 Safety Requirements 16
5.3 Security Requirements 16
6. Acceptance Criteria 17
7. Accelerated Development Plan (Feb–March) 18
Team Assumptions 18
Sprint 1 (Feb 1–14): Persistence & Local Transactions 18
Sprint 2 (Feb 15–28): Routing & Distributed Execution 18
Sprint 3 (Mar 1–14): Cross-Shard Coordination 19
Sprint 4 (Mar 15–31): Replication, Failover & Validation 19
8. Appendices 19
8.1 Transaction Examples (Corrected Semantics) 19
8.2 Traceability Matrix 20

Revision History

Name	Date	Reason For Changes	Version
Base	21/01/2026	Base version(No changes)	1.0
Version 2	02/02/2026	● Tentative System architecture ● Better UML Diagrams ● Updated Sharding Techniques ● Documented naming convention ● Clarified sharding logic and other details ● Clearly defined dataflow and system interactions	2.0

Naming Convention (REQ-XXX):

General Format: REQ-<CATEGORY>-<SEQUENCE>

Category Naming Rules

Functional

Category	Meaning	Example
TX	Transaction processing	REQ-TX-001
DATA	Data management / storage	REQ-DATA-002
REP	Replication & fault tolerance	REQ-REP-004
COORD	Distributed coordination	REQ-COORD-001

Non-Functional

Category	Meaning	Example
PERF	Performance	REQ-PERF-002
SAFE	Safety / reliability	REQ-SAFE-001

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements of a distributed transaction engine designed to provide horizontal scalability through sharding, strong consistency via atomic transaction processing, and fault tolerance through data replication.

The system draws conceptual inspiration from Google Spanner and CockroachDB.

1.2 Document Conventions

The **Distributed Sharded Transaction Engine** enables atomic money transfers across distributed shards with ACID-like guarantees.

Key Features:

- Horizontal data partitioning across multiple shards
- Atomic cross-shard transfers using two-phase commit (2PC)
- Fault tolerance via primary-follower replication
- Durability through write-ahead logging
- Automatic failure detection and recovery

1.3 System Components

The system consists of the following components (per UML diagram):

- **API Gateway**: Receives and validates transaction requests
- **Transaction Coordinator**: Orchestrates 2PC protocol
- **Shard Map:** JSON file on coordinator mapping partitions to shards
- **Shard**: Contains primary database or replicas managing the account subset
- **Write Ahead Log**: Records operations before execution
- **Kafka Log**: Distributed transaction log for fast data streaming
- **Load Monitor:** Monitors load on shards continuously and performs partition migration when required every few weeks

1.4 Definitions and Acronyms

Term	Definition
Shard	**Horizontal partition of data containing a subset of accounts.**
Replica	**Copy of shard data (PRIMARY or FOLLOWER).**
WAL	**Write-Ahead Log ensuring durability.**
2PC	**Two-Phase Commit protocol for atomic distributed transactions.**
Coordinator	**Components orchestrating transactions and shard directing.**

1.5 Product Scope

The product is intended for applications that require real-time digital payments and transaction processing, serving as a gateway for high-speed financial operations in domains such as e-commerce and banking systems.

The primary objective of the product is to provide a low-latency, highly reliable, and scalable transaction platform that ensures secure and fast payment processing and bank transfers.

2. Overall Description

2.1 Transaction Processing

REQ-TX-001: Transaction Submission

- System SHALL accept transactions with: transaction ID (UUID), source account, destination account, and amount
- API Gateway SHALL validate: positive amount, distinct accounts, valid UUIDs, Rate Limiting
- Invalid transactions SHALL be rejected with appropriate error codes

REQ-TX-002: Single-Shard Transactions

- For transactions where both accounts reside on same shard, system SHALL:
    - Execute locally without 2PC overhead
    - Write operations to WAL before applying to shard
    - Update both account balances atomically
    - Replicate to follower replicas

REQ-TX-003: Cross-Shard Transactions

- For transactions spanning multiple shards, Transaction Coordinator SHALL execute 2PC:
    - **Prepare Phase**: Send PREPARE to all involved shards, wait for PREPARED/ABORT responses
    - **Commit Phase**: If all shards are prepared, send COMMIT; otherwise, send ABORT
    - Publish final decision to Kafka Log for durability
- Cross-shard transactions SHALL be atomic (all shards commit or all abort)

REQ-TX-004: Transaction Status

- System SHALL track transaction states: PENDING, PREPARED, COMMITTED, ABORTED
- Status transitions: PENDING → PREPARED → COMMITTED, or PENDING/PREPARED → ABORTED

2.2 Data Management

REQ-DATA-001: Sharding

- The Coordinator SHALL deterministically map each account to a logical shard partition using a hash-based or modulo partitioning function.
- All entries belonging to the same account SHALL always map to the same logical shard partition.

- The mapping of logical shard partitions to physical shard servers SHALL be maintained by the Shard Map and MAY change dynamically to support load balancing and rebalancing.
- System SHALL support a configurable number of shards
- Each shard SHALL by default have 1 leader and 2 followers/replicas.

REQ-DATA-002: Write-Ahead Logging

- For each write operation, the Shard Leader SHALL append the operation to its Write-Ahead Log (WAL) before applying the operation to the ledger state.
- The Shard Leader SHALL replicate WAL entries to all replica shards, and each Replica SHALL persist the received WAL entry to stable storage before acknowledging the Leader.
- A write operation SHALL be considered committed only after the Shard Leader receives acknowledgements from the required number of Replicas.
- WAL entries SHALL contain at minimum: log ID, transaction ID, operation type, and timestamp.
- All WAL writes SHALL be synchronously flushed to disk (fsync) on both the Shard Leader and Replicas before a commit is acknowledged.
- The WAL SHALL support crash recovery through log replay and rollback of uncommitted entries.

REQ-DATA-003: Load Balancing

- The Shard Map SHALL maintain the mapping of logical shard partitions to physical shard servers.
- The Transaction Coordinator SHALL query the Shard Map to resolve the target physical shard for each transaction prior to execution.
- The Shard Load Monitor SHALL continuously monitor shard load metrics and detect imbalances across shard servers.
- Upon detecting imbalance or overload, the Shard Load Monitor SHALL initiate controlled shard partition rebalancing by coordinating with the Transaction Coordinator.
- Shard rebalancing SHALL be performed at the granularity of logical shard partitions and SHALL ensure transactional consistency by temporarily halting affected partitions during migration.
- Shard rebalancing SHOULD minimize impact on transaction latency and system availability but SHALL NOT require global system quiescence.

2.3 Replication and Fault Tolerance

REQ-REP-001: Primary-Follower Architecture

- Each Shard SHALL have one PRIMARY database and multiple FOLLOWER (atleast 2) replicas
- PRIMARY handles all write operations
- FOLLOWERS replicate PRIMARY's WAL entries

REQ-REP-002: Replication Protocol

- PRIMARY SHALL replicate log entries to FOLLOWERS
- PRIMARY SHALL wait for acknowledgment from the majority of replicas before committing (incase of 2 replicas, it will wait for ack from atleast 1)

REQ-REP-003: Failure Detection

- Failure Detector SHALL send periodic heartbeats to all replicas (default: 5-second interval)
- Replica SHALL be marked failed after missing multiple consecutive heartbeats (default: 3 misses)
- Failure Detector SHALL notify Replica Manager of failures

REQ-REP-004: Primary Failover

- When PRIMARY fails, Replica Manager SHALL:
    - Select the most up-to-date FOLLOWER
    - Promote FOLLOWER to PRIMARY role
    - Update Shard Manager's shard registry
    - Notify the system of the new primary
- New PRIMARY SHALL replay WAL to ensure a consistent state

2.4 Distributed Coordination

REQ-COORD-001: Kafka Log

- The Kafka Log SHALL organize records into named topics based on transaction categories or domains.
- Each topic partition SHALL assign monotonically increasing, sequential offsets to log entries.
- The Kafka Log SHALL guarantee total ordering of entries within each partition.
- The Kafka Log SHALL durably persist log entries before acknowledging write completion to the producer.
- Kafka consumer groups SHALL be used to distribute transaction processing across multiple Coordinators.

REQ-COORD-002: Transaction Recovery

- Transaction Coordinators SHALL operate as stateless consumers within a Kafka consumer group.
- In the event of a Coordinator failure, Kafka SHALL reassign the affected topic partitions to another Coordinator instance.
- The newly assigned Coordinator SHALL resume processing from the last committed Kafka offset.
- Transaction execution SHALL be idempotent such that reprocessing of Kafka records does not result in duplicate or inconsistent state changes.
- Shard-level write-ahead logging and replication SHALL ensure durability and consistency independent of Coordinator failures.

3. Non-Functional Requirements

3.1 Performance

REQ-PERF-001: System SHALL support a minimum of 100 transactions/second per shard
REQ-PERF-002: Single-shard transaction latency < 50ms (p99)
REQ-PERF-003: Cross-shard transaction latency < 200ms (p99)
REQ-PERF-004: System SHALL scale horizontally with additional shards

3.2 Reliability and Safety

REQ-SAFE-001: System SHALL maintain invariants:

- Total money conserved (sum of balances constant)
- No negative balances
- All committed transactions are durable

REQ-SAFE-002: System SHALL tolerate:

- Loss of any single replica without data loss
- Primary failure with automatic failover
- Process crashes with full recovery via WAL replay

REQ-SAFE-003: Committed transactions SHALL survive single node crashes

3.3 Quality Attributes

REQ-QUAL-001: Code SHALL be well-documented with inline comments explaining design decisions
REQ-QUAL-002: System SHALL include comprehensive tests:

- Unit tests for each component
- Integration tests for transaction flows
- Fault injection tests for failure scenarios
- Correctness tests demonstrating atomicity

REQ-QUAL-003: As an educational system, code SHALL prioritize readability over micro-optimizations

4. System Architecture

4.1 Component Interactions

Transaction Flow (Cross-Shard):

1. Transaction Flow (Cross-Shard)
2. The API Gateway SHALL receive the transaction request, perform validation, and publish the transaction to the Kafka Log.
3. Transaction Coordinators SHALL consume transactions from Kafka as part of a consumer group and operate without maintaining persistent transaction state.
4. For each transaction, the Coordinator SHALL consult the Shard Map to identify all logical shard partitions involved.
5. The Coordinator SHALL send a PREPARE request to each involved shard.
6. Each Shard SHALL validate the transaction locally and append a PREPARE record to its Write-Ahead Log without applying state changes.
7. Each Shard SHALL respond with either PREPARED or ABORT.
8. If any Shard responds with ABORT, the Coordinator SHALL send an ABORT request to all previously prepared shards.
9. If all Shards respond with PREPARED, the Coordinator SHALL send a COMMIT request directly to the involved shards.
10. Upon receiving COMMIT, each Shard SHALL apply the transaction, replicate the committed WAL entry to followers, and make the update durable.
11. Transaction execution SHALL be idempotent to tolerate Coordinator failure and Kafka record reprocessing.

4.2 Data Flow

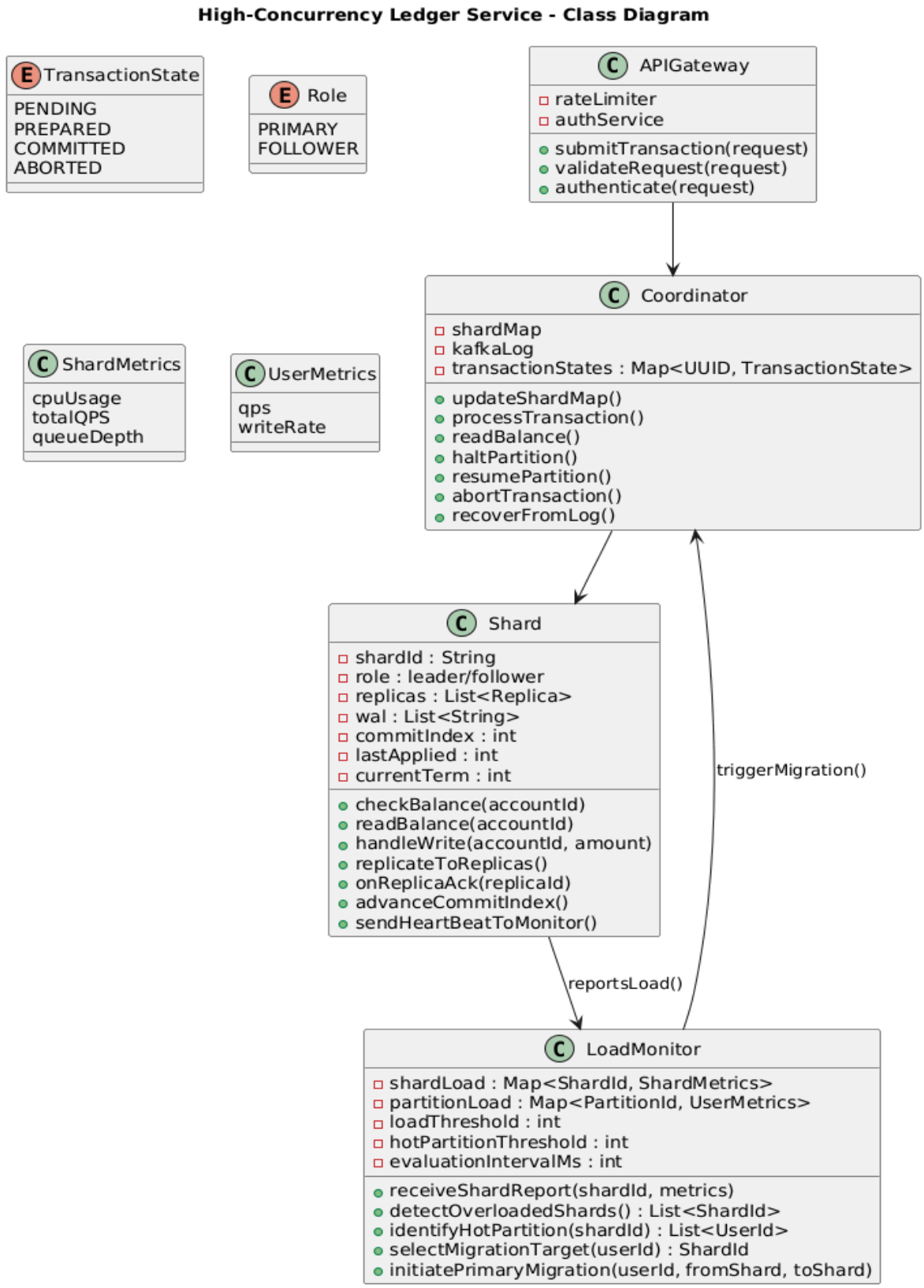
- **Read/Write Path:** API Gateway → Kafka Log → Coordinator → Shard Leader → Followers
- **Recovery Path**: Kafka Log → Coordinator (new consumer) → Shards (idempotent execution + WAL recovery)

4.3 Component Responsibilities

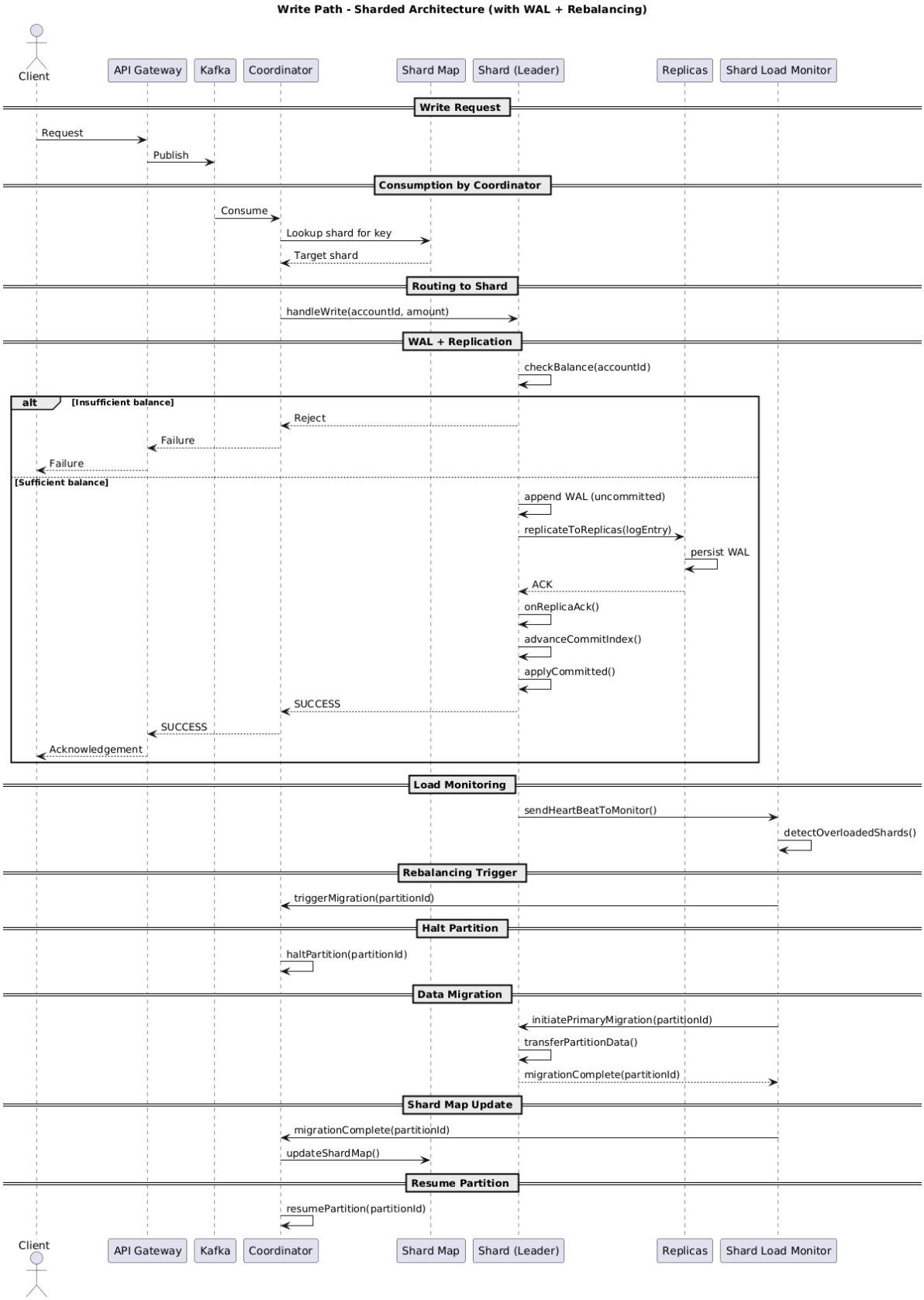
Component	Key Responsibilities
API Gateway	● Request validation ● UUID generation ● Publishing transactions to the Kafka Log
Transaction Coordinator	● Stateless consumption of transactions from Kafka ● Shard lookup via Shard Map ● Routing of requests to appropriate shards ● Coordination of multi-shard ● Transaction execution without maintaining persistent transaction state
Shard	● Local transaction validation and execution ● Write-ahead logging ● Commit ● Decision enforcement ● Coordination of replication to follower replicas
Replica	● Persistence of replicated write-ahead log entries ● Application of committed log entries ● Maintenance of follower shard state.
Write Ahead Log	● Durable persistence of shard-level operations ● Support for crash recovery via ● Log replay ● Rollback of uncommitted entries
Kafka Log	● Durable ● Ordered event stream for transaction ingestion and distribution ● Providing fan-out and load balancing across Transaction Coordinators
Load Monitor	● Continuous monitoring of shard load metrics ● Initiation of controlled shard partition rebalancing to maintain balanced load across shard servers

4.4 UML Diagrams

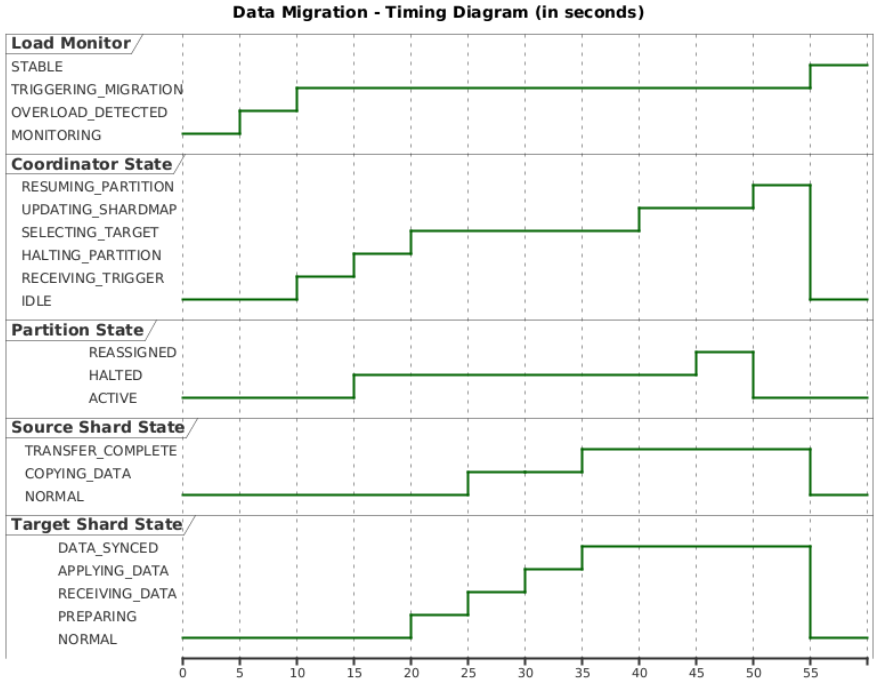
Class Diagram

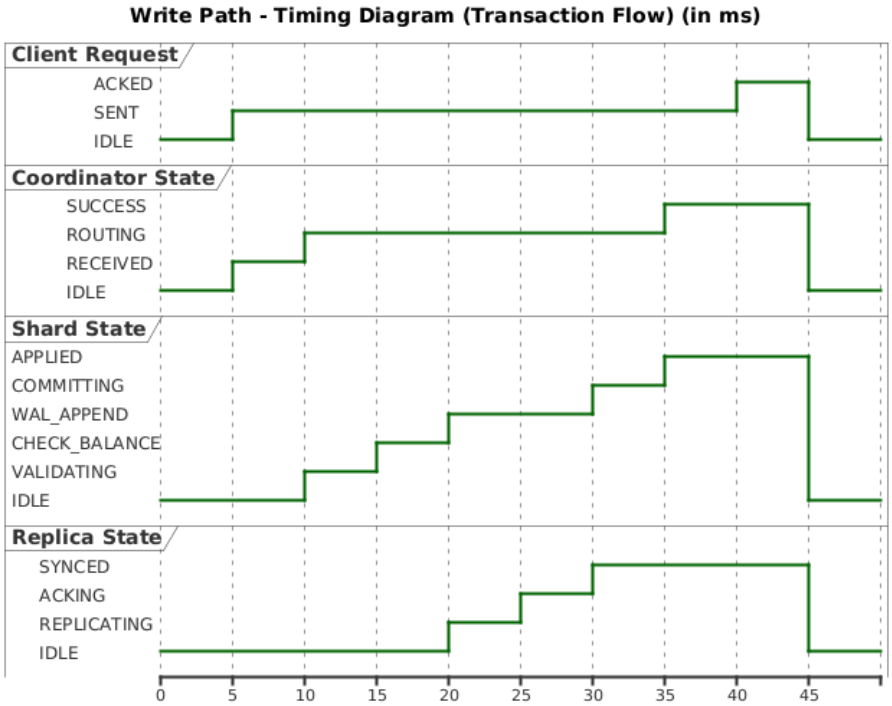


Sequence Diagram

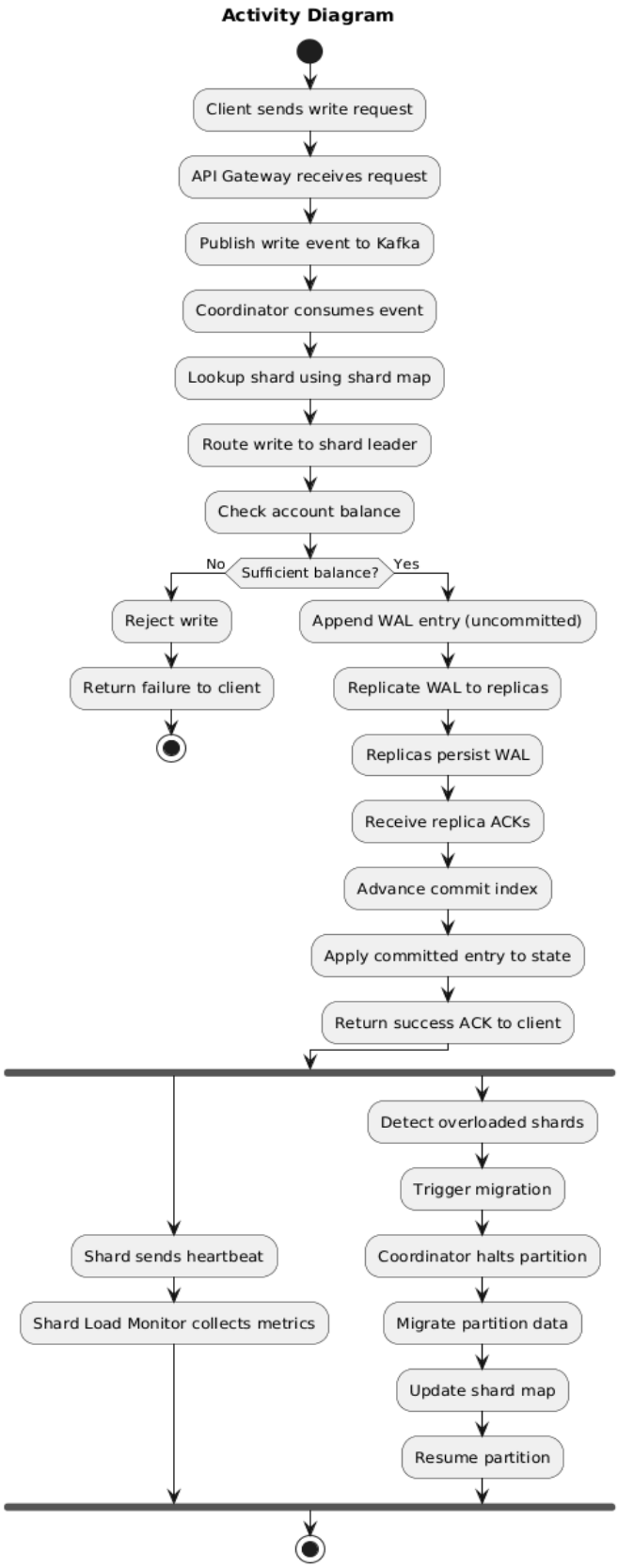


Timing Diagrams

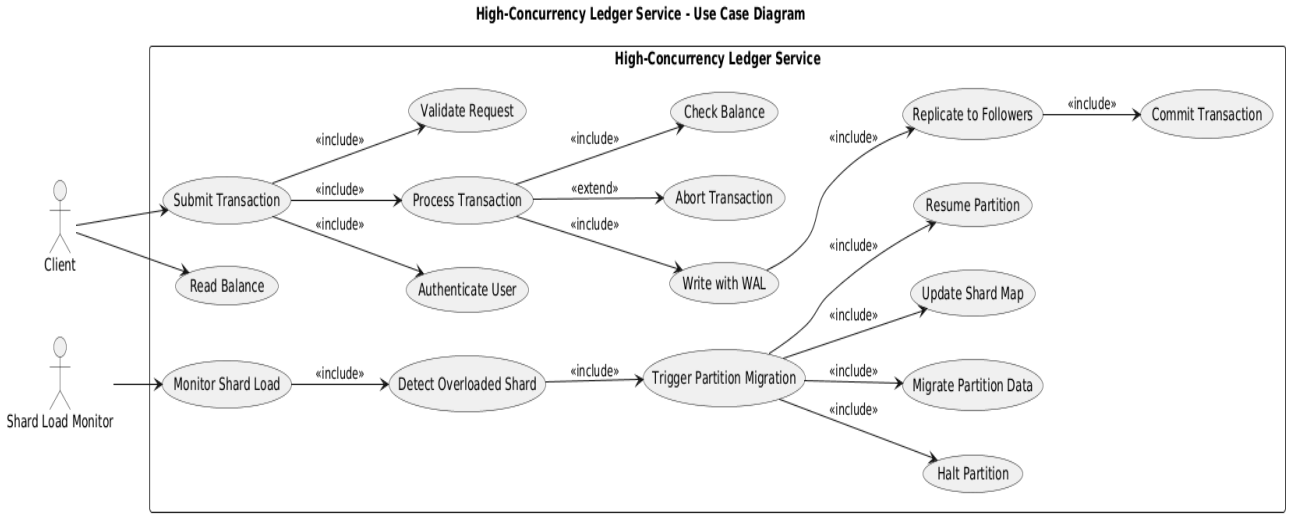




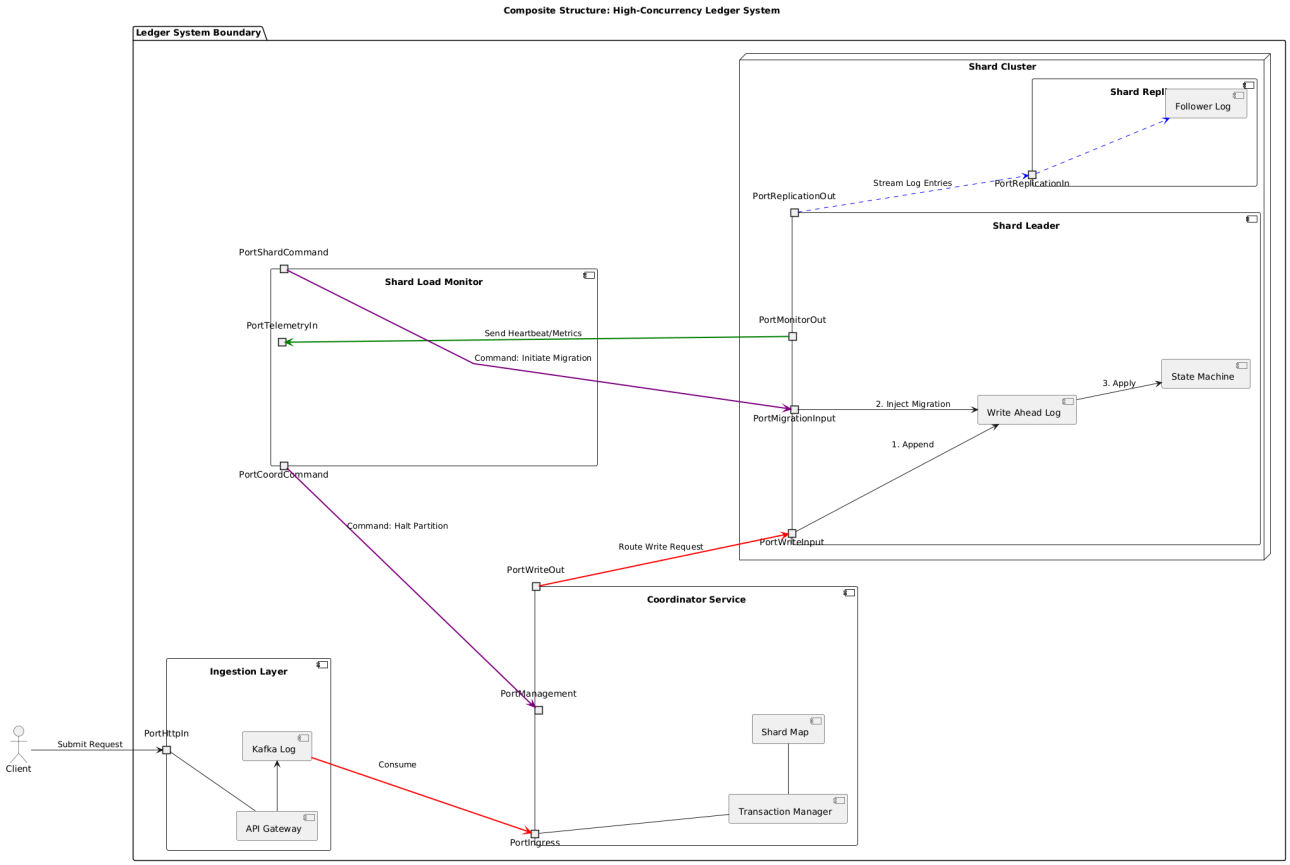
Activity Diagram



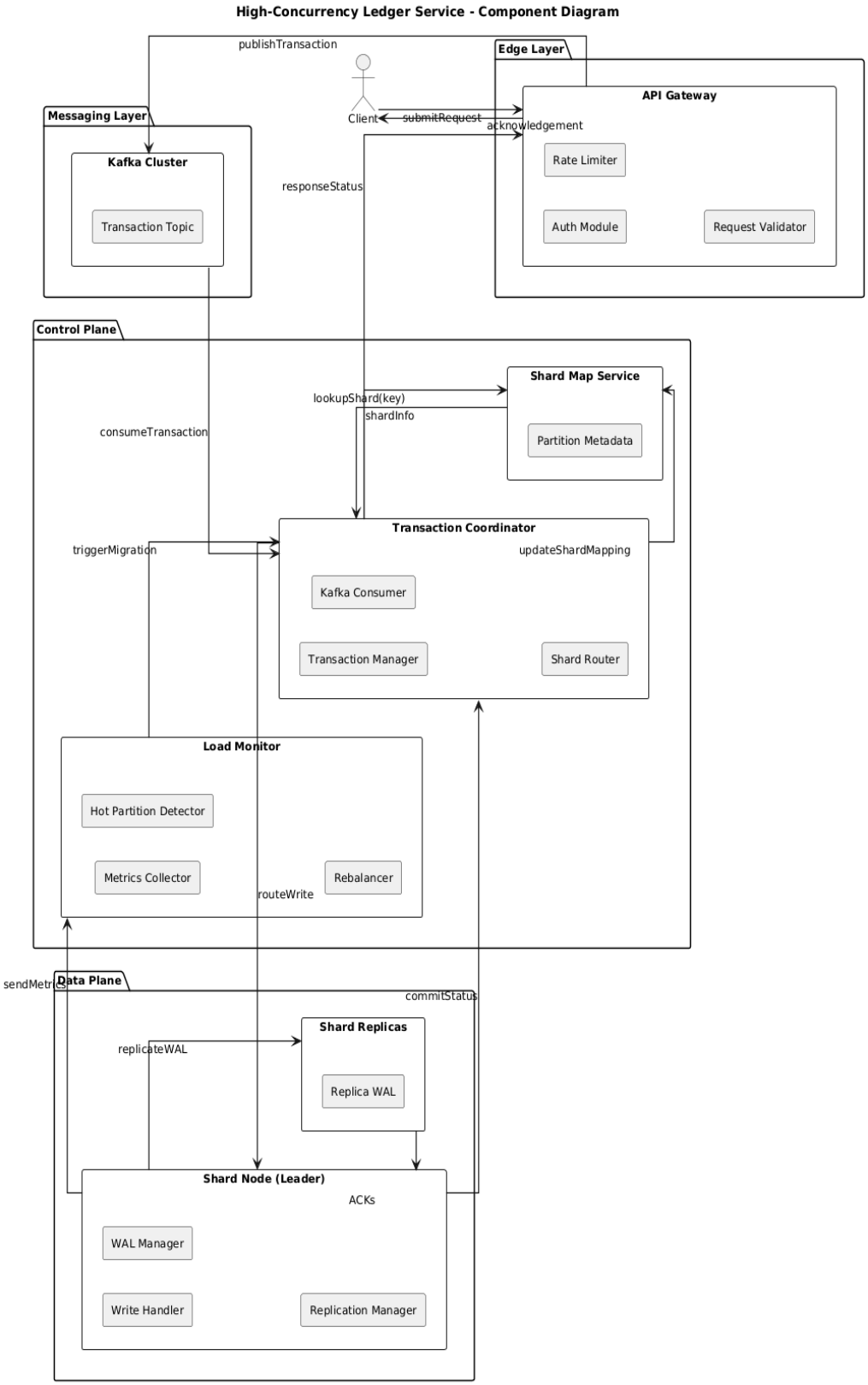
Use Case Diagram



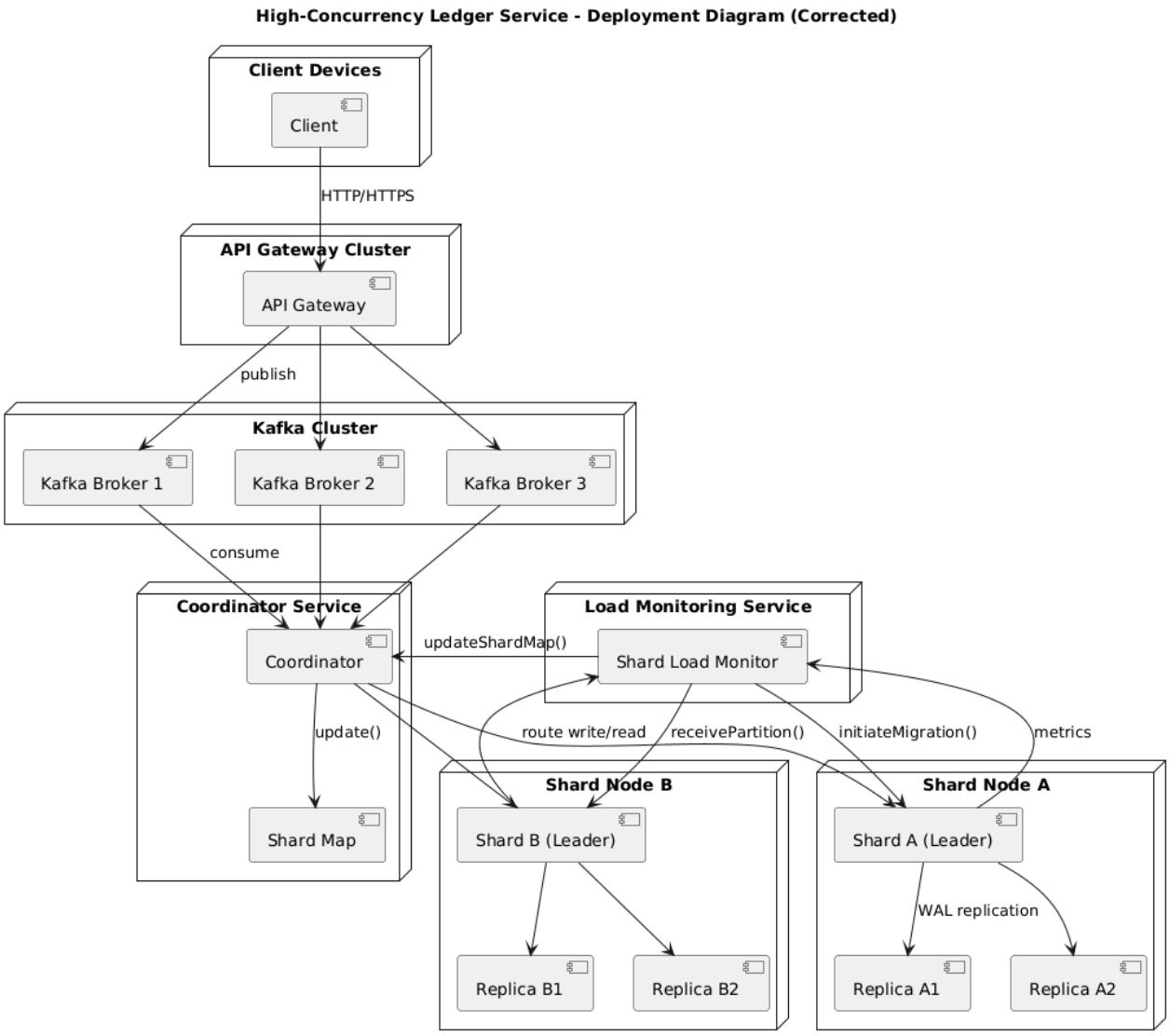
Composite Diagram



Component Diagram



Deployment Diagram



5. Constraints and Assumptions

5.1 Performance Requirements

- The system SHALL run on commodity hardware.
- The system SHALL operate over standard TCP/IP networking.
- The system SHALL use the local filesystem for persistence and SHALL NOT require specialized storage hardware or devices.

5.2 Safety Requirements

- The system SHALL prioritize correctness, consistency, and clarity of behavior over raw performance.
- The system SHALL follow well-established distributed systems design patterns.
- The system SHALL maintain clear separation of responsibilities between components.
- The system SHALL minimize reliance on external dependencies to reduce operational complexity.

5.3 Security Requirements

- The network SHALL be asynchronous but eventually reliable.
- Node failures SHALL follow a fail-stop model, and Byzantine failures are explicitly out of scope.
- Disk storage SHALL follow a crash-recovery model and is assumed not to exhibit silent data corruption.
- The system SHALL be deployed within a single datacenter and SHALL NOT provide geo-distributed consistency guarantees.
- Clock drift between nodes SHALL be bounded, and no global time oracle or TrueTime-style service is required.

6. Acceptance Criteria

The system is considered complete when:

1. **Functional Correctness**:
    - Single-shard transactions execute successfully
    - Cross-shard transactions SHALL preserve consistency
    - Failures do not corrupt data or create partial commits
2. **Fault Tolerance**:
    - The system SHALL recover from shard leader (primary replica) failures without data loss.
    - The system SHALL tolerate Transaction Coordinator failures through Kafka consumer reassignment and reprocessing.
    - Write-ahead log replay SHALL restore shards to a consistent state after crash recovery.
3. **Documentation**:
    - All components have clear documentation
    - UML diagrams (class, sequence, deployment) provided
    - Test cases demonstrate correctness
4. **Testing**:
    - 70% unit test coverage
    - Integration tests for all transaction types
    - Fault injection tests pass
    - Performance benchmarks meet targets

7. Accelerated Development Plan (Feb–March)

Team Assumptions

- Team size: 4 members
- Timeline: 8 weeks (4 sprints)
- Constraint: Correctness, durability, and recoverability are prioritized over performance optimizations.

Sprint 1 (Feb 1–14): Persistence & Local Transactions

Focus: Single-node correctness with WAL-backed execution.

Responsibilities

- M1: Ledger structures and deterministic account-to-partition mapping
- M2: WAL format, fsync semantics, and replay-based recovery
- M3: Shard execution logic (atomic debit/credit)
- M4: Crash recovery and invariant tests (no negative balances)

Deliverables

- A functional single-shard node that survives crashes
- Deterministic account-to-partition mapping

Sprint 2 (Feb 15–28): Routing & Distributed Execution

Focus: Stateless coordination and shard routing.

Responsibilities

- M1: Stateless Transaction Coordinator (Kafka consumer, shard lookup, routing)
- M2: API Gateway (validation, UUID generation, Kafka publishing)
- M3: Shard-side RPC or messaging interface
- M4: Routing and multi-shard execution tests

Deliverables

- Coordinators route transactions to correct shards

- End-to-end happy-path execution across multiple shards

Sprint 3 (Mar 1–14): Cross-Shard Coordination

Focus: Coordinated multi-shard execution using shard-local WAL and idempotency.

Responsibilities

- M1: Cross-shard execution flow (PREPARE / COMMIT messages, no global state)
- M2: Shard-side prepared state handling and rollback logic
- M3: Kafka integration for ingress ordering and coordinator fan-out
- M4: Fault injection during coordinator and shard failures

Deliverables

- Consistent cross-shard execution under coordinator failures
- No partial shard commits after failures

Sprint 4 (Mar 15–31): Replication, Failover & Validation

Focus: High availability and submission readiness.

Responsibilities

- M1: Primary-follower replication and commit rules
- M2: Shard leader election and primary promotion
- M3: WAL-based shard recovery and follower catch-up
- M4: Final acceptance testing and documentation

Deliverables

- System tolerates single shard node failure without data loss
- Full traceability against requirements

8. Appendices

8.1 Transaction Examples (Corrected Semantics)

Example 1: Single-Shard Success
Transfer $100 from AccountA to AccountB (same shard)
Shard validates, writes WAL, applies atomically
Result: COMMITTED

Example 2: Cross-Shard Success
Transfer $50 from AccountA (Shard1) to AccountC (Shard2)
Both shards PREPARE and then COMMIT
Result: COMMITTED

Example 3: Cross-Shard Abort
Transfer $1000 from AccountA (balance $100) to AccountC
Shard1 ABORTS, prepared shards roll back
Result: ABORTED

(Note: Cross-shard consistency is enforced via shard-level WAL and idempotent execution.)

8.2 Traceability Matrix

Requirement ID	Requirement Title	Priority	Type	Design Component(s)	Verification Method	Acceptance Criteria	Status
REQ-TX-001	Transaction Submission	High	Functional	API Gateway; Transaction Coordinator	Unit + Integration Test	Invalid inputs rejected; valid transactions forwarded	Not Started
REQ-TX-002	Single-Shard Transaction	High	Functional	Shard; WAL; Ledger; Replica	Integration Test	Atomic execution; WAL flushed before apply;	Not Started

						replicated to followers	
REQ-TX-003	Cross-Shard Transaction	High	Functional	Transaction Coordinator; Shards; Kafka Log	Integration + Fault Injection	All shards commit or abort consistently	Not Started
REQ-TX-004	Transaction Status Tracking	High	Functional	Transaction Coordinator; Kafka Log	Unit Test	Valid state transitions maintained	Not Started
REQ-DATA-002	Account Ledger Integrity	High	Functional	Ledger	Unit + Correctness Test	No negative balances; atomic debit/credit	Not Started
REQ-DATA-003	Write-Ahead Logging	Moderate	Functional	WAL; Replica	Crash Recovery Test	WAL fsync before apply; recovery restores consistent state	Not Started
REQ-REP-001	Primary-Follower Replication		Functional	Replica; Shard	Integration Test	Majority acknowledgment required	Not Started

						before commit	
REQ-REP-002	Replication Protocol		Functional	Primary Replica; Followers	Integration Test	Followers apply log entries in order	Not Started
REQ-REP-003	Failure Detection		Functional	Failure Detector	Fault Injection Test	Replica marked failed after missed heartbeats	Not Started
REQ-REP-004	Primary Failover		Functional	Replica Manager;	Failover Simulation	Most up-to-date follower promoted; WAL replayed	Not Started
REQ-COORD-001	Kafka Log Durability		Functional	Kafka Log	Integration Test	Entries persisted before acknowledgment; ordered offsets	Not Started
REQ-COORD-002	Transaction Recovery		Functional	Transaction Coordinator; Kafka Log	Crash Recovery Test	Incomplete transactions resolved correctly	Not Started